

Appendix: Epilog Ruleset

Dependent Type Checking in Epilog

Bryce Goldman

```
1  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
2  % Calculus of Constructions
3  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4  %%% Implementing a dependent type checker in Epilog
5  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
6
7  % This file provides the Epilog rules implementing the backend for a CoC
8  % typechecker, with support for a simple dependent type theory. Aspirationally,
9  % this should facilitate automated theorem proving similar to other CoC-based
10 % proof assistants such as Lean, Rocq, and Agda.
11 %
12 % Due to the limited scope of the project in its current form, the system lacks
13 % many of the language features needed to fully realize this goal, but it does
14 % support type resolution and inference for the fundamental constructs of
15 % Martin-Löf intuitionistic type theory, modulo equality types (see Future Work);
16 % although the grammar appears to omit dependent sums, these are still supported
17 % indirectly through a combination of inductive (sum) types and struct (record)
18 % types.
19 %
20 % The input to the typechecker, a top-level module, consists of a list of
21 % declarations, described by the following grammar:
22 % Declaration ::= Axiom | Def | Inductive | Struct
23 % Axiom       ::= axiom Name : Term
24 % Def         ::= def Name .{UniverseParams}? Param* : Term := Term
25 % Inductive   ::= inductive Name .{UniverseParams}? Param* : Term where Constructor+
26 % Struct      ::= struct Name .{UniverseParams}? Param* : Term where Field+
27 %
28 % UniverseParams ::= Name | Name, UniverseParams
29 % Param          ::= (Name : Term)
30 % Constructor    ::= "/" Param
31 % Field          ::= Param
32 %
33 % Term ::= Term.Name
34 %       | match Term with Arm+
35 %       | Term -> Term
36 %       | Term Term
37 %       | Name
38 %       | Sort Universe
39 %       | Type Nat
40 %       | Type
41 %       | Prop
42 %       | lambda Param . Term
43 %       | Pi Param . Term
44 %       | (Term)
45 %
46 % Arm      ::= "/" Pattern => Term
47 % Pattern  ::= Name | _ | Name(Patterns)
48 % Patterns ::= <eps> | Pattern | Pattern, Patterns
49 % Universe ::= Nat | Name | max(Universe, Universe) | Universe + Nat | (Universe)
50 %
```

```

51 % Typechecking proceeds according to standard CoC algorithms, leveraging the
52 % native unification and pattern-matching faculties of Epilog. In particular,
53 % we are able to circumvent the implementation of a heavy-duty type solver by
54 % simply allowing Epilog to syntactically unify the structure unknown type
55 % metavariables. This approach has proven largely successful, and the current
56 % state of the typechecker is able to resolve many common universe-polymorphic
57 % patterns (see ./tests), failing on only the more sophisticated universe
58 % constraints.
59 %
60 %% TODO: Future work
61 % - There are some bugs in the universe polymorphism for inductive and struct
62 % types. In particular, the solver fails when multiple universe variables are
63 % bound by an inductive/struct declaration, but not all of them are consumed
64 % by every constructor/field. This is likely because Epilog's unifier gets
65 % stuck trying to unify the absent metavariables to resolve the type of a
66 % constructor/projection.
67 %
68 % - Currently the system lacks support for propositional equality types, which
69 % hinders its ability to fully serve as a theorem prover. A natural next step
70 % would be to implement equality types with explicit refl introduction and the
71 % usual transformation rules (via symmetry/transitivity). The main challenge
72 % is ensuring that the def_eq rule suffices to support Eq-intro, as this
73 % requires fully unrolling definitions to test for judgemental equality.
74
75 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
76 %% References
77 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
78 %% Epilog and Logic Programming:
79 % - http://epilog.stanford.edu/documentation/epilogjs/guide.php
80 % - http://logicprogramming.stanford.edu/public/index.php
81 %% Dependent Type Theory
82 % - https://www.danielgratzer.com/papers/type-theory-book.pdf
83 %% Calculus of Constructions
84 % - https://inria.hal.science/hal-01094195v1/document
85 % - https://sdiehl.github.io/typechecker-zoo/coc/calculus-of-constructions.html
86
87 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
88 %% Misc. Utilities
89 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
90
91 %% Predicates used as helper relations throughout the ruleset, mostly pertaining
92 % to operations on generic sets/lists.
93
94 %% Checks that a list is empty.
95 empty(nil)
96
97 %% append_elt(Xs, Y, Zs) appends element Y to the list Xs to produce the list Zs.
98 append_elt(nil, Y, [Y])
99 append_elt(cons(X, Xs), Y, cons(X, Zs)) :- append_elt(Xs, Y, Zs)
100
101 %% rev(Xs, Ws) reverses the list Xs to produce the list Ws.
102 rev(nil, nil)
103 rev(cons(X, Xs), Ws) :- rev(Xs, Zs) & append_elt(Zs, X, Ws)
104
105 %% in(X, Ys) is true when X is an element of the list Ys.
106 in(X, cons(X, Ys))
107 in(X, cons(Y, Ys)) :- distinct(X, Y) & in(X, Ys)
108
109 %% union(Xs, Ys, Zs) is true when Zs is the union of sets (lists) Xs and Ys.
110 union(nil, Ys, Ys)
111 union(cons(X, Xs), Ys, cons(X, Zs)) :- ~in(X, Ys) & union(Xs, Ys, Zs)

```

```

112
113
114 %%%%%%%%%%%%%%
115 %% Structure
116 %%%%%%%%%%%%%%
117
118 %% Predicates that check the structure of constructs throughout the ruleset.
119 % A number of these predicates are currently vestigial, but are left in the
120 % ruleset intentionally to document the internal representation of the
121 % corresponding object.
122
123 %% Enumerate the structure of a term.
124 is_term(var(X))
125 is_term(app(Fun, Arg)) :- is_term(Fun) & is_term(Arg)
126 is_term(abs(X, Ty, Body)) :- is_term(Ty) & is_term(Body)
127 is_term(pi(X, Ty, Body)) :- is_term(Ty) & is_term(Body)
128 is_term(usort(Uni)) :- is_universe(Uni)
129 is_term(match(Target, Arms)) :- is_term(Target) & all_arms(Arms)
130 is_term(constructor(Name, Args)) :- all_terms(Args)
131 is_term(proj(Term, Field)) :- is_term(Term)
132
133 %% Predicates to check what category a given term belongs to.
134 is_var(var(X))
135 is_app(app(Fun, Arg))
136 is_abs(abs(X, Ty, Body))
137 is_pi(pi(X, Ty, Body))
138 is_usort(usort(Uni))
139 is_match(match(Target, Arms))
140 is_constructor(constructor(Name, Args))
141 is_proj(proj(Term, Field))
142
143 %% Checks the structure of a universe.
144 is_universe(uconst(N))
145 is_universe(uvar(X))
146 is_universe(uadd(U, N)) :- is_universe(U)
147 is_universe(umax(U, V)) :- is_universe(U) & is_universe(V)
148
149 %% Predicates to check what category a given universe belongs to.
150 is_uconst(uconst(N)) :- number(N)
151 is_uvar(uvar(X))
152 is_uadd(uadd(U, N)) :- is_universe(U)
153 is_umax(umax(U, V)) :- is_universe(U) & is_universe(V)
154
155 %% Predicates to check what category a meta-theoretic construct belongs to.
156 is_parameter(parameter(Name, Ty)) :- is_term(Ty)
157
158 is_constr_decl(constr_decl(Name, Ty)) :- is_term(Ty)
159
160 is_field(field(Name, Ty)) :- is_term(Ty)
161
162 is_arm(match.arm(Pattern, Body)) :- is_pattern(Pattern) & is_term(Body)
163
164 is_pattern(pattern.var(X))
165 is_pattern(pattern.constructor(X, Patterns)) :- all_patterns(Patterns)
166 is_pattern(pattern.wildcard)
167
168 is_uni_param(universe.parameter(Name))
169
170 all_terms(nil)
171 all_terms(cons(Term, Rest)) :- is_term(Term) & all_terms(Rest)
172

```

```

173 all_uni(nil)
174 all_uni(cons(Uni, Rest)) :- is_universe(Uni) & all_uni(Rest)
175
176 all_params(nil)
177 all_params(cons(Param, Rest)) :- is_parameter(Param) & all_params(Rest)
178
179 all_constr_decls(nil)
180 all_constr_decls(cons(Constr, Rest)) :- is_constr_decl(Constr) & all_constr_decls(Rest)
181
182 all_fields(nil)
183 all_fields(cons(Field, Rest)) :- is_field(Field) & all_fields(Rest)
184
185 all_arms(nil)
186 all_arms(cons(Arm, Rest)) :- is_arm(Arm) & all_arms(Rest)
187
188 all_patterns(nil)
189 all_patterns(cons(Pattern, Rest)) :- is_pattern(Pattern) & all_patterns(Rest)
190
191 all_uni_param(nil)
192 all_uni_param(const(Uni_Param, Rest)) :- is_uni_param(Uni_Param) & all_uni_param(Rest)
193
194 %% Predicates to check what category a given declaration belongs to.
195 is_axiom(axiom(Name, Uni_Params, Ty)) :- all_uni_param(Uni_Params) & is_term(Ty)
196
197 is_def(def(Name, Uni_Params, Params, Ty, Body)) :- all_uni_param(Uni_Params) &
198     all_params(Params) & is_term(Ty) & is_term(Body)
199
200 is_induct(inductive(Name, Uni_Params, Params, Ty, Constrs)) :- all_uni_param(Uni_Params) &
201     all_params(Params) & is_term(Ty) & all_constr_decls(Constrs)
202
203 is_struct(struct(Name, Uni_Params, Params, Ty, Fields)) :- all_uni_param(Uni_Params) &
204     all_params(Params) & is_term(Ty) & all_fields(Fields)
205
206
207 %%%%%%%%%%%
208 %% Universes
209 %%%%%%%%%%%
210
211 %% Handlers for universe objects. In the case of universe polymorphism these may
212 % be represented by metavariables, which are solved for during the type
213 % inference process. To facilitate universe solving, we reduce each universe
214 % to a normal form, descend the structural tree of the involved terms, and
215 % unify universes that are constrained to be equal (currently we do not
216 % implement support for more complex relationships between universes).
217
218 %% Reduce the given universe to its normal form. Constants are simplified
219 % statically, so that the resulting normal form universe cannot (in general) be
220 % reduced further.
221 %
222 %% TODO: at the moment iteration uadds are not folded into a single uadd, but
223 % this pattern is permitted by the grammar. It is possible this could
224 % allow the unifier to get stuck on constraints that are tractable, if
225 % the user does something stupid like ((u + 1) + 1).
226 universe.normalize(uadd(uconst(N), M), uconst(R)) :- evaluate(plus(N, M), R)
227 universe.normalize(uadd(U, N), uadd(U, N)) :- is_universe(U) & ~is_uconst(U)
228 universe.normalize(umax(U, V), uconst(R)) :- universe.normalize(U, uconst(N)) &
229     universe.normalize(V, uconst(M)) & evaluate(max(N, M), R)
230 universe.normalize(umax(U, V), umax(U, V)) :- universe.normalize(U, Nf_U) &
231     universe.normalize(V, Nf_V) & ~is_uconst(Nf_U)
232 universe.normalize(umax(U, V), umax(U, V)) :- universe.normalize(U, Nf_U) &
233     universe.normalize(V, Nf_V) & ~is_uconst(Nf_V)

```

```

234 universe.normalize(U, U) :- is_universe(U) & ~is_uadd(U) & ~is_umax(U)
235
236 %% Unify a pair of universes, forcing them to be structurally equal using the
237 % native unification process of Epilog.
238 universe.unify(uconst(N), uconst(N))
239 universe.unify(uvar(X), uvar(X))
240 universe.unify(uadd(U, N), uconst(M)) :- evaluate(minus(M, N), R) & leq(0, R) & universe.unify(U,
    ↪ uconst(R))
241 universe.unify(uconst(M), uadd(U, N)) :- evaluate(minus(M, N), R) & leq(0, R) & universe.unify(U,
    ↪ uconst(R))
242 universe.unify(uadd(U, N), uadd(V, N)) :- universe.unify(U, V)
243 universe.unify(umax(U1, V1), umax(U2, V2)) :- universe.unify(U1, U2) & universe.unify(V1, V2)
244
245 %% Unify all universe metavariables found in a pair of terms. These terms must
246 % be otherwise syntactically equal for this to succeed.
247 %
248 %% NOTE: Terms should be in de Bruijn form before solving for universes.
249 universe.solve(var(X), var(X))
250 universe.solve(app(Fun1, Arg1), app(Fun2, Arg2)) :- universe.solve(Fun1, Fun2) & universe.solve(Arg1,
    ↪ Arg2)
251 universe.solve(abs(X, Ty1, Body1), abs(X, Ty2, Body2)) :- universe.solve(Ty1, Ty2) &
    ↪ universe.solve(Body1, Body2)
252 universe.solve(pi(X, Ty1, Body1), pi(X, Ty2, Body2)) :- universe.solve(Ty1, Ty2) & universe.solve(Body1,
    ↪ Body2)
253 universe.solve(usort(U), usort(V)) :- universe.unify(U, V)
254 universe.solve(usort(U), usort(V)) :- universe.unify(V, U)
255 universe.solve(match(Target1, Arms1), match(Target2, Arms2)) :- universe.solve(Target1, Target2) &
    ↪ universe.solve_arms(Arms1, Arms2)
256 universe.solve(constructor(Name1, Args1), constructor(Name2, Args2)) :- universe.solve_args(Args1, Args2)
257 universe.solve(proj(Term1, Field1), proj(Term2, Field2)) :- universe.solve(Term1, Term2)
258
259 universe.solve_arms(nil, nil)
260 universe.solve_arms(cons(match.arm(Pattern1, Body1), Rest1), cons(match.arm(Pattern2, Body2), Rest2)) :-
    universe.solve(Body1, Body2) & universe.solve_arms(Rest1, Rest2)
261
262
263 universe.solve_args(nil, nil)
264 universe.solve_args(cons(Arg1, Rest1), cons(Arg2, Rest2)) :- universe.solve(Arg1, Arg2) &
    ↪ universe.solve_args(Rest1, Rest2)
265
266 %%%%%%%%%%%%%%
267 %%% Contexts
268 %%%%%%%%%%%%%%
269
270 %% Map Interface
271 % Implements basic operations on the map data structure, which is fundamental
272 % to the context construct used throughout typechecking. Maps are implemented
273 % as a list of elements of the form map.entry(Key, Value).
274
275 %% Insert the key-value pair (New_Key, New_Val) into the given map, to produce
276 % a new map.
277 map.insert(nil, New_Key, New_Val, [map.entry(New_Key, New_Val)])
278 map.insert(cons(map.entry(New_Key, Val), Map), New_Key, New_Val, cons(map.entry(New_Key, New_Val), Map))
279 map.insert(cons(map.entry(Key, Val), Map), New_Key, New_Val, cons(map.entry(Key, Val), New_Map)) :-
    distinct(Key, New_Key) & map.insert(Map, New_Key, New_Val, New_Map)
280
281
282 %% Find the value associated with the given key in the map (if it exists).
283 map.lookup(cons(map.entry(Key, Val), Map), Key, Val)
284 map.lookup(cons(map.entry(Other_Key, Other_Val), Map), Key, Val) :- map.lookup(Map, Key, Val)
285
286 %% Check whether a map contains a particular key.
287 map.contains(Map, Key) :- map.lookup(Map, Key, Val)
288

```

```

289 %% Context Interface
290 % Implements basic operations on the context data structure. This consists of
291 % four maps:
292 % - Var_Map: maps variable names to their types
293 % - Constr_Map: maps constructors to their function types
294 % - Axiom_Map: maps axiom names to their types
295 % - Def_Map: maps definitions names to sub-structures of the form
296 %             ctx.def(Uni_Params, Ty), where Ty is the definition's type and
297 %             Uni_Params are the definition's universe parameters. Definition
298 %             declarations without universe parameters are stored as axioms
299 %             for simplicity.
300
301 %% Var_Map Interface
302 ctx.lookup_var(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Var, Term) :- map.lookup(Var_Map, Var, Term)
303 ctx.extend_var(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Var, Term, ctx(New_Var_Map, Constr_Map,
304   ↪ Axiom_Map, Def_Map)) :-
305   map.insert(Var_Map, Var, Term, New_Var_Map)
306 ctx.extend_vars(Ctx, cons(parameter(Name, Ty), Rest), New_Ctx) :- ctx.extend_var(Ctx, Name, Ty,
307   ↪ Inter_Ctx) &
308   ctx.extend_vars(Inter_Ctx, Rest, New_Ctx)
309 ctx.contains_var(Ctx, Var) :- ctx.lookup_var(Ctx, Var, Term)
310
311 %% Constr_Map Interface
312 ctx.lookup_constr(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Name, Term) :- map.lookup(Constr_Map,
313   ↪ Name, Term)
314 ctx.extend_constr(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Name, Term, ctx(Var_Map, New_Constr_Map,
315   ↪ Axiom_Map, Def_Map)) :-
316   map.insert(Constr_Map, Name, Term, New_Constr_Map)
317 ctx.contains_constr(Ctx, Name) :- ctx.lookup_constr(Ctx, Name, Term)
318
319 %% Axiom_Map Interface
320 ctx.lookup_axiom(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Axiom, Term) :- map.lookup(Axiom_Map,
321   ↪ Axiom, Term)
322 ctx.extend_axiom(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Axiom, Term, ctx(Var_Map, Constr_Map,
323   ↪ New_Axiom_Map, Def_Map)) :-
324   map.insert(Axiom_Map, Axiom, Term, New_Axiom_Map)
325 ctx.contains_axiom(Ctx, Axiom) :- ctx.lookup_axiom(Ctx, Axiom, Term)
326
327 %% Def_Map Interface
328 ctx.lookup_def(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Name, Def) :- map.lookup(Def_Map, Name, Def)
329 ctx.extend_def(ctx(Var_Map, Constr_Map, Axiom_Map, Def_Map), Name, Uni_Params, Ty, ctx(Var_Map,
330   ↪ Constr_Map, Axiom_Map, New_Def_Map)) :-
331   map.insert(Def_Map, Name, ctx.def(Uni_Params, Ty), New_Def_Map)
332 ctx.contains_def(Ctx, Name) :- ctx.lookup_def(Ctx, Name, Def)
333
334 %% Builds an empty context.
335 ctx.nil(ctx(nil, nil, nil, nil))
336
337 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
338 %%% Substitution
339 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
340
341 %% Utilities for performing term/variable substitutions. Throughout typechecking
342 % and type inference applications, definition-inlining, and universe freshening
343 % require replacing occurrences of one element in a structure with some other
344 % element.
345
346 %% Replaces all occurrences of the universe variable "u" with Substitution["u"],
347 % for each key "u" in the Substitution map.
348 uni_subst_all(usort(U), Substitution, usort(New_U)) :- uni_subst(U, Substitution, New_U)

```

```

344 %% Each other case simply descends on the structure tree of the term.
345 uni_subst_all(var(X), Substitution, var(X))
346 uni_subst_all(app(Fun, Arg), Substitution, app(Sub_Fun, Sub_Arg)) :- uni_subst_all(Fun, Substitution,
    ↪ Sub_Fun) &
347     uni_subst_all(Arg, Substitution, Sub_Arg)
348 uni_subst_all(abs(X, Ty, Body), Substitution, abs(X, Sub_Ty, Sub_Body)) :- uni_subst_all(Ty,
    ↪ Substitution, Sub_Ty) &
349     uni_subst_all(Body, Substitution, Sub_Body)
350 uni_subst_all(pi(X, Ty, Body), Substitution, pi(X, Sub_Ty, Sub_Body)) :- uni_subst_all(Ty, Substitution,
    ↪ Sub_Ty) &
351     uni_subst_all(Body, Substitution, Sub_Body)
352 uni_subst_all(proj(Term, Field), Substitution, proj(Sub_Term, Field)) :- uni_subst_all(Term,
    ↪ Substitution, Sub_Field)
353 uni_subst_all(Term, Substitution, Term) :- ~is_usort(Term) & ~is_var(Term) & ~is_app(Term) &
    ↪ ~is_abs(Term) & ~is_pi(Term) & ~is_proj(Term)
354
355 %% Perform the substitution for each universe variable to be substituted.
356 uni_subst(uvar(U), Substitution, New_U) :- map.lookup(Substitution, U, New_U)
357 uni_subst(uvar(U), Substitution, uvar(U)) :- ~map.contains(Substitution, U)
358 uni_subst(uconst(N), Substitution, uconst(N))
359 uni_subst(uadd(U, N), Substitution, uadd(New_U, N)) :- uni_subst(U, Substitution, New_U)
360 uni_subst(umax(U, V), Substitution, umax(New_U, New_V)) :- uni_subst(U, Substitution, New_U) &
    ↪ uni_subst(V, Substitution, New_V)
361
362 %% Checks whether variable X is bound in the Bound list in substitute_with_context.
363 is_bound(cons(X, Xs), X)
364 is_bound(cons(Y, Ys), X) :- distinct(Y, X) & is_bound(Ys, X)
365
366 %% fresh_var(X, Zs, Y) generates a new variable name Y with prefix X by appending
367 % each element of Zs to X (separated by a _ character). In practice, the list
368 % Zs should contain all other variable names we seek to avoid. This is not the
369 % most elegant way to accomplish this, but because we lack a state we cannot
370 % simply update some global counter to create a fresh name.
371 fresh_var(X, nil, Y) :- evaluate(stringappend(X, "_"), Y)
372 fresh_var(X, cons(Z, Zs), Y) :- fresh_var(X, Zs, W) & evaluate(stringappend(W, Z), Y)
373
374 %% Checks whether the name X occurs free (i.e. is not bound by a lambda/pi) in
375 % the given term.
376 occurs_free(Y, var(Y))
377 occurs_free(X, app(Fun, Arg)) :- occurs_free(X, Fun)
378 occurs_free(X, app(Fun, Arg)) :- occurs_free(X, Arg)
379 occurs_free(X, abs(Y, Ty, Body)) :- occurs_free(X, Ty)
380 occurs_free(X, abs(Y, Ty, Body)) :- distinct(X, Y) & occurs_free(X, Body)
381 occurs_free(X, pi(Y, Ty, Body)) :- occurs_free(X, Ty)
382 occurs_free(X, pi(Y, Ty, Body)) :- distinct(X, Y) & occurs_free(X, Body)
383 occurs_free(X, match(Target, Arms)) :- occurs_free(X, Target)
384 occurs_free(X, match(Target, Arms)) :- occurs_free_arms(X, Arms)
385 occurs_free(X, constructor(Name, Args)) :- occurs_free_args(X, Args)
386 occurs_free(X, proj(Term, Target)) :- occurs_free(X, Term)
387
388 occurs_free_arms(X, cons(match.arm(Pattern, Body), Rest)) :- occurs_free(X, Body)
389 occurs_free_arms(X, cons(match.arm(Pattern, Body), Rest)) :- occurs_free_arms(X, Rest)
390
391 occurs_free_args(X, cons(Arg, Rest)) :- occurs_free(X, Arg)
392 occurs_free_args(X, cons(Arg, Rest)) :- occurs_free_args(X, Rest)
393
394 %% Replaces the name of one variable with a new name.
395 rename_var(Prev, New, var(Prev), var(New))
396 rename_var(Prev, New, var(X), var(X)) :- distinct(Prev, X)
397 rename_var(Prev, New, app(Fun, Arg), app(R_Fun, R_Arg)) :- rename_var(Prev, New, Fun, R_Fun) &
    ↪ rename_var(Prev, New, Arg, R_Arg)
398 rename_var(Prev, New, abs(Prev, Ty, Body), abs(New, R_Ty, Body)) :- rename_var(Prev, New, Ty, R_Ty)

```

```

399 rename_var(Prev, New, abs(X, Ty, Body), abs(X, R_Ty, R_Body)) :- distinct(Prev, X) & rename_var(Prev,
    ↪ New, Ty, R_Ty) &
400     rename_var(Prev, New, Body, R_Body)
401 rename_var(Prev, New, pi(Prev, Ty, Body), pi(Prev, R_Ty, Body)) :- rename_var(Prev, New, Fun, R_Fun)
402 rename_var(Prev, New, pi(X, Ty, Body), pi(X, R_Ty, R_Body)) :- distinct(Prev, X) & rename_var(Prev, New,
    ↪ Ty, R_Ty) &
403     rename_var(Prev, New, Body, R_Body)
404 rename_var(Prev, New, match(Target, Arms), match(R_Target, R_Arms)) :- rename_var(Prev, New, Target,
    ↪ R_Target) &
405     rename_var_arms(Prev, New, Arms, R_Arms)
406 rename_var(Prev, New, proj(Term, Field), match(R_Term, Field)) :- rename_var(Prev, New, Term, R_Term)
407 rename_var(Prev, New, Term, Term) :- ~is_var(Term) & ~is_app(Term) & ~is_abs(Term) & ~is_pi(Term) &
    ↪ ~is_match(Term) & ~is_proj(Term)
408
409 rename_var_arms(Prev, New, nil, nil)
410 rename_var_arms(Prev, New, cons(match.arm(Pattern, Body), Rest), cons(match.arm(Pattern, Body), R_Rest))
    ↪ :-
411     pattern_binds_var(Pattern, Prev) & rename_var_arms(Prev, New, Rest, R_Rest)
412 rename_var_arms(Prev, New, cons(match.arm(Pattern, Body), Rest), cons(match.arm(Pattern, R_Body),
    ↪ R_Rest)) :-
413     ~pattern_binds_var(Pattern, Prev) & rename_var(Prev, New, Body, R_Body) & rename_var_arms(Prev, New,
    ↪ Rest, R_Rest)
414
415 %% Perform a context-sensitive substitution in a term by replacing all instances
416 % of X with Src. If X is bound in the current context, it is not substituted
417 % (to avoid variable capture under a lambda/pi).
418 substitute_with_context(X, Src, var(X), Bound, Src) :- ~is_bound(Bound, X)
419 substitute_with_context(X, Src, var(X), Bound, var(X)) :- is_bound(Bound, X)
420 substitute_with_context(X, Src, var(Y), Bound, var(Y)) :- distinct(X, Y)
421
422 substitute_with_context(X, Src, app(Fun, Arg), Bound, app(Sub_Fun, Sub_Arg)) :-
    ↪ substitute_with_context(X, Src, Fun, Bound, Sub_Fun) &
423     substitute_with_context(X, Src, Arg, Bound, Sub_Arg)
424
425 substitute_with_context(X, Src, abs(X, Ty, Body), Bound, abs(X, Sub_Ty, Body)) :-
    ↪ substitute_with_context(X, Src, Ty, Bound, Sub_Ty)
426 substitute_with_context(X, Src, abs(Y, Ty, Body), Bound, abs(Fresh, Sub_Ty, Sub_Body)) :- distinct(X, Y)
    ↪ &
427     occurs_free(Y, Src) & fresh_var(Y, Bound, Fresh) & rename_var(Y, Fresh, Body, New_Body) &
428     substitute_with_context(X, Src, Ty, cons(Fresh, Bound), Sub_Ty) & substitute_with_context(X, Src,
    ↪ New_Body, cons(Fresh, Bound), Sub_Body)
429 substitute_with_context(X, Src, abs(Y, Ty, Body), Bound, abs(Y, Sub_Ty, Sub_Body)) :- distinct(X, Y) &
430     ~occurs_free(Y, Src) & substitute_with_context(X, Src, Ty, cons(Y, Bound), Sub_Ty) &
    ↪ substitute_with_context(X, Src, Body, cons(Y, Bound), Sub_Body)
431
432 substitute_with_context(X, Src, pi(X, Ty, Body), Bound, pi(X, Sub_Ty, Body)) :-
    ↪ substitute_with_context(X, Src, Ty, Bound, Sub_Ty)
433 substitute_with_context(X, Src, pi(Y, Ty, Body), Bound, pi(Fresh, Sub_Ty, Sub_Body)) :- distinct(X, Y) &
434     occurs_free(Y, Src) & fresh_var(Y, Bound, Fresh) & rename_var(Y, Fresh, Body, New_Body) &
435     substitute_with_context(X, Src, Ty, cons(Fresh, Bound), Sub_Ty) & substitute_with_context(X, Src,
    ↪ New_Body, cons(Fresh, Bound), Sub_Body)
436 substitute_with_context(X, Src, pi(Y, Ty, Body), Bound, pi(Y, Sub_Ty, Sub_Body)) :- distinct(X, Y) &
437     ~occurs_free(Y, Src) & substitute_with_context(X, Src, Ty, cons(Y, Bound), Sub_Ty) &
438     substitute_with_context(X, Src, Body, cons(Y, Bound), Sub_Body)
439
440 substitute_with_context(X, Src, match(Target, Arms), Bound, match(Sub_Target, Sub_Arms)) :-
441     substitute_with_context(X, Src, Target, Bound, Sub_Target) & substitute_arms_with_context(X, Src,
    ↪ Arms, Bound, Sub_Arms)
442
443 substitute_with_context(X, Src, proj(Term, Field), Bound, proj(Sub_Term, Field)) :-
    ↪ substitute_with_context(X, Src, Term, Bound, Sub_Term)
444

```

```

445 substitute_with_context(X, Src, Tgt, Bound, Tgt) :- ~is_var(Tgt) & ~is_app(Tgt) & ~is_abs(Tgt) &
    ↳ ~is_pi(Tgt) & ~is_match(Tgt) & ~is_proj(Tgt)
446
447 substitute_arms_with_context(X, Src, nil, Bound, nil)
448 substitute_arms_with_context(X, Src, cons(match.arm(Pattern, Body), Rest), Bound, cons(match.arm(Pattern,
    ↳ Body), Sub_Rest)) :-
449     pattern_binds_var(Pattern, X) & substitute_arms_with_context(X, Src, Rest, Bound, Sub_Rest)
450 substitute_arms_with_context(X, Src, cons(match.arm(Pattern, Body), Rest), Bound, cons(match.arm(Pattern,
    ↳ Sub_Body), Sub_Rest)) :-
451     ~pattern_binds_var(Pattern, X) & substitute_with_context(X, Src, Body, Bound, Sub_Body) &
    ↳ substitute_arms_with_context(X, Src, Rest, Bound, Sub_Rest)
452
453 pattern_binds_var(pattern.var(X), X)
454 pattern_binds_var(pattern.constructor(Name, Patterns), X) :- pattern_binds_var_any(Patterns, X)
455
456 pattern_binds_var_any(cons(Pattern, Rest), X) :- pattern_binds_var(Pattern, X)
457 pattern_binds_var_any(cons(Pattern, Rest), X) :- pattern_binds_var_any(Rest, X)
458
459 %% Substitute Src for each instance of X in Tgt.
460 substitute(X, Src, Tgt, Result) :- substitute_with_context(X, Src, Tgt, nil, Result)
461
462
463 %%%%%%%%%%%%%%%
464 %% Equivalence
465 %%%%%%%%%%%%%%%
466
467 %% Handlers for term normalization, alpha-equivalence, and unification.
468
469 %% Convert variable names of the given term to de Bruijn indices. Required to
470 % make a pair of equivalent lambda/pi terms syntactically equal.
471 alpha_normalize_rec(var(X), Db_Ctx, N, var(M)) :- map.lookup(Db_Ctx, X, M)
472 alpha_normalize_rec(var(X), Db_Ctx, N, var(X)) :- ~map.contains(Db_Ctx, X)
473 alpha_normalize_rec(app(Fun, Arg), Db_Ctx, N, app(Db_Fun, Db_Arg)) :- alpha_normalize_rec(Fun, Db_Ctx, N,
    ↳ Db_Fun) &
474     alpha_normalize_rec(Arg, Db_Ctx, N, Db_Arg)
475 alpha_normalize_rec(abs(X, Ty, Body), Db_Ctx, N, abs(M, Db_Ty, Db_Body)) :- evaluate(plus(N, 1), M) &
    map.insert(Db_Ctx, X, M, Ext_Db_Ctx) & alpha_normalize_rec(Ty, Ext_Db_Ctx, M, Db_Ty) &
    ↳ alpha_normalize_rec(Body, Ext_Db_Ctx, M, Db_Body)
476
477 alpha_normalize_rec(pi(X, Ty, Body), Db_Ctx, N, pi(M, Db_Ty, Db_Body)) :- evaluate(plus(N, 1), M) &
    map.insert(Db_Ctx, X, M, Ext_Db_Ctx) & alpha_normalize_rec(Ty, Ext_Db_Ctx, M, Db_Ty) &
    ↳ alpha_normalize_rec(Body, Ext_Db_Ctx, M, Db_Body)
478
479 alpha_normalize_rec(usort(U), Db_Ctx, N, usort(U))
480 alpha_normalize_rec(proj(Term, Field), Db_Ctx, N, proj(Db_Term, Field)) :- alpha_normalize_rec(Term,
    ↳ Db_Ctx, N, Db_Term)
481 alpha_normalize_rec(match(Target, Arms), Db_Ctx, N, match(Db_Target, Db_Arms)) :-
    ↳ alpha_normalize_rec(Target, Db_Ctx, N, Db_Target) &
482     alpha_normalize_arms(Arms, Db_Ctx, N, Db_Arms)
483 alpha_normalize_rec(ctor(Name, Args), Db_Ctx, N, ctor(Name, Db_Args)) :-
    ↳ alpha_normalize_args(Args, Db_Ctx, N, Db_Args)
484 alpha_normalize_rec(Term, Db_Ctx, N, Term) :- ~is_var(Term) & ~is_app(Term) & ~is_abs(Term) &
    ↳ ~is_pi(Term) &
485     ~is_usort(Term) & ~is_match(Term) & ~is_constructor(Term) & ~is_proj(Term)
486
487 alpha_normalize(Term, Db_Term) :- alpha_normalize_rec(Term, nil, 0, Db_Term)
488
489 %% Check whether two terms are alpha-equivalent (i.e. are syntactically equal up
490 % to renaming bound variables).
491 alpha_equiv(Term1, Term2) :- alpha_normalize(Term1, Db) & alpha_normalize(Term2, Db)
492
493 %% Term normalization. Currently performs beta-reduction on lambda types,
494 % eta-reduction on lambda types, and unfolds definitions.

```

```

495 normalize(app(Fun, Arg), Ctx, app(Nf_Fun, Nf_Arg)) :- normalize(Fun, Ctx, Nf_Fun) & normalize(Arg, Ctx,
    ↪ Nf_Arg) & ~is_abs(Nf_Fun)
496 normalize(app(Fun, Arg), Ctx, Nf) :- normalize(Fun, Ctx, abs(Nf_X, Nf_Ty, Nf_Body)) & normalize(Arg, Ctx,
    ↪ Nf_Arg) & substitute(Nf_X, Nf_Arg, Nf_Body, Nf)
497
498 normalize(abs(X, Ty, Body), Ctx, Nf_Fun) :- normalize(Ty, Ctx, Nf_Ty) & ctx.extend_var(Ctx, X, Nf_Ty,
    ↪ Ext_Ctx) &
499     normalize(Body, Ext_Ctx, app(Nf_Fun, var(Nf_Var))) & ~occurs_free(Nf_Var, Nf_Fun)
500 normalize(abs(X, Ty, Body), Ctx, abs(X, Nf_Ty, Nf_Body)) :- normalize(Ty, Ctx, Nf_Ty) &
501     ctx.extend_var(Ctx, X, Nf_Ty, Ext_Ctx) & normalize(Body, Ext_Ctx, Nf_Body) & ~is_app(Nf_Body)
502 normalize(abs(X, Ty, Body), Ctx, abs(X, Nf_Ty, app(Nf_Fun, var(Nf_Var)))) :- normalize(Ty, Ctx, Nf_Ty) &
503     ctx.extend(Ctx, X, Nf_Ty, Ext_Ctx) & normalize(Body, Ext_Ctx, app(Nf_Fun, var(Nf_Var))) &
    ↪ occurs_free(Nf_Var, Nf_Fun)
504
505 normalize(pi(X, Ty, Body), Ctx, pi(X, Nf_Ty, Nf_Body)) :- normalize(Ty, Ctx, Nf_Ty) &
506     ctx.extend_var(Ctx, X, Nf_Ty, Ext_Ctx) & normalize(Body, Ext_Ctx, Nf_Body)
507
508 normalize(var(X), Ctx, Def) :- ctx.lookup_axiom(Ctx, X, Def)
509 normalize(var(X), Ctx, var(X)) :- ~ctx.contains_axiom(Ctx, X)
510
511 normalize(Term, Ctx, Term) :- ~is_app(Term) & ~is_abs(Term) & ~is_pi(Term) & ~is_var(Term)
512
513 %% Checks whether the inputs are judgementally equal in the provided context.
514 % Terms are normalized both before and after unification, in case new redexes
515 % are introduced during unification.
516 def_eq(Term1, Term2, Ctx) :- normalize(Term1, Ctx, Nf1) & normalize(Term2, Ctx, Nf2) & unify(Nf1, Nf2) &
    ↪ normalize(Nf1, Ctx, Nff1) & normalize(Nf1, Ctx, Nff2) & alpha_equiv(Nff1, Nff2)
517
518
519
520 %% During alpha conversion for terms containing metavariables (i.e. pre-unification
521 % terms), we need to traverse the term's structure tree in BFS order to guarantee
522 % termination. However, Epilog executes rules in DFS order, which causes the
523 % unifier to get stuck when alpha-converting terms in preparation for unification.
524 %
525 % Our somewhat hacky solution to this problem is to enforce a maximum search
526 % depth. This guarantees that each structural form is compared to the term,
527 % provided the term's structure tree is sufficiently shallow. In practice a
528 % relatively small maximum depth seems to suffice, but this may need to be
529 % increased to resolve the types of more sophisticated terms. Modify max_depth
530 % below to adjust this traversal depth.
531
532 max_depth(4)
533 iter_depth(Depth, Succ_Depth) :- evaluate(plus(Depth, 1), Succ_Depth) & max_depth(Max) & leq(Succ_Depth,
    ↪ Max)
534
535 %% Same as alpha_normalize_rec, but with BFS using the bounded depth constraint
536 % described above.
537 alpha_bounded_normalize_rec(var(X), Db_Ctx, N, var(M), Depth) :- map.lookup(Db_Ctx, X, M)
538 alpha_bounded_normalize_rec(var(X), Db_Ctx, N, var(X), Depth) :- ~map.contains(Db_Ctx, X)
539 alpha_bounded_normalize_rec(app(Fun, Arg), Db_Ctx, N, app(Db_Fun, Db_Arg), Depth) :- iter_depth(Depth,
    ↪ Succ_Depth) &
540     alpha_bounded_normalize_rec(Fun, Db_Ctx, N, Db_Fun, Succ_Depth) & alpha_bounded_normalize_rec(Arg,
    ↪ Db_Ctx, N, Db_Arg, Succ_Depth)
541 alpha_bounded_normalize_rec(abs(X, Ty, Body), Db_Ctx, N, abs(M, Db_Ty, Db_Body), Depth) :-
    ↪ iter_depth(Depth, Succ_Depth) &
542     evaluate(plus(N, 1), M) & map.insert(Db_Ctx, X, M, Ext_Db_Ctx) & alpha_bounded_normalize_rec(Ty,
    ↪ Ext_Db_Ctx, M, Db_Ty, Succ_Depth) &
543     alpha_bounded_normalize_rec(Body, Ext_Db_Ctx, M, Db_Body, Succ_Depth)
544 alpha_bounded_normalize_rec(pi(X, Ty, Body), Db_Ctx, N, pi(M, Db_Ty, Db_Body), Depth) :-
    ↪ iter_depth(Depth, Succ_Depth) &
545     evaluate(plus(N, 1), M) & map.insert(Db_Ctx, X, M, Ext_Db_Ctx) & alpha_bounded_normalize_rec(Ty,
    ↪ Ext_Db_Ctx, M, Db_Ty, Succ_Depth) &

```

```

546     alpha_bounded_normalize_rec(Body, Ext_Db_Ctx, M, Db_Body, Succ_Depth)
547 alpha_bounded_normalize_rec(usort(U), Db_Ctx, N, usort(U), Depth)
548 alpha_bounded_normalize_rec(proj(Term, Field), Db_Ctx, N, proj(Db_Term, Field), Depth) :-
549   ↪ iter_depth(Depth, Succ_Depth) &
550     alpha_bounded_normalize_rec(Term, Db_Ctx, N, Db_Term, Succ_Depth)
551 alpha_bounded_normalize_rec(match(Target, Arms), Db_Ctx, N, match(Db_Target, Db_Arms), Depth) :-
552   ↪ iter_depth(Depth, Succ_Depth) &
553     alpha_bounded_normalize_rec(Target, Db_Ctx, N, Db_Target, Succ_Depth) &
554     ↪ alpha_bounded_normalize_arms(Arms, Db_Ctx, N, Db_Arms, Succ_Depth)
555 alpha_bounded_normalize_rec(ctor(Name, Args), Db_Ctx, N, ctor(Name, Db_Args), Depth) :-
556   ↪ iter_depth(Depth, Succ_Depth) &
557     alpha_bounded_normalize_args(Args, Db_Ctx, N, Db_Args, Succ_Depth)
558 alpha_bounded_normalize_rec(Term, Db_Ctx, N, Term, Depth) :- ~is_var(Term) & ~is_app(Term) &
559   ↪ ~is_abs(Term) & ~is_pi(Term) &
560     ↪ ~is_usort(Term) & ~is_match(Term) & ~is_constructor(Term) & ~is_proj(Term)
561
562 alpha_bounded_normalize_arms(nil, Db_Ctx, N, nil, Depth)
563 alpha_bounded_normalize_arms(cons(match.arm(Pattern, Body), Rest), Db_Ctx, N, cons(match.arm(Db_Pattern,
564   ↪ Db_Body), Db_Rest), Depth) :- alpha_bounded_normalize_rec(Body, Db_Ctx, N, Db_Body, Depth) &
565   ↪ alpha_bounded_normalize_arms(Rest, Db_Ctx, N, Db_Rest, Depth)
566
567 alpha_bounded_normalize_args(nil, Db_Ctx, N, nil, Depth)
568 alpha_bounded_normalize_args(cons(Arg, Rest), Db_Ctx, N, cons(Db_Arg, Db_Rest), Depth) :-
569   ↪ alpha_bounded_normalize_rec(Arg, Db_Ctx, N, Db_Arg, Depth) & alpha_bounded_normalize_args(Rest,
570   ↪ Db_Ctx, N, Db_Rest, Depth)
571
572 alpha_bounded_normalize(Term, Db_Term) :- alpha_bounded_normalize_rec(Term, nil, 0, Db_Term, 0)
573
574 %% Unify a pair of terms, by performing (bounded) alpha-conversion and solving
575 % for any remaining universe metavariables, if possible.
576 unify(Term1, Term2) :- alpha_bounded_normalize(Term1, Db_Term1) & alpha_bounded_normalize(Term2,
577   ↪ Db_Term2) & universe.solve(Db_Term1, Db_Term2)
578 unify(Term1, Term2) :- alpha_bounded_normalize(Term2, Db_Term2) & alpha_bounded_normalize(Term1,
579   ↪ Db_Term1) & universe.solve(Db_Term2, Db_Term1)
580
581 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
582 %% Type Checking
583 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
584
585 %% Predicates for facilitating declaration and term type-checking. Interacts
586 % heavily with the type inference mechanisms below (most terms must have their
587 % types *inferred* since only function types are annotated with explicit type
588 % information).
589
590 %% Checks if a term is a type.
591 is_type(usort(U)) :- is_universe(U)
592
593 %% Constructs a pi type from a vector of parameters and a body type.
594 % e.g. make_pi_type([(x : A), (y : B)], C, ?) unifies ? to Pi (x : A). Pi (y : B). C
595 make_pi_type(nil, Body_Ty, Body_Ty)
596 make_pi_type(cons(parameter(Name, Ty), Rest), Body_Ty, pi(Name, Ty, Acc_Ty)) :- make_pi_type(Rest,
597   ↪ Body_Ty, Acc_Ty)
598
599 %% Constructs a lambda term from a vector of parameters and a body.
600 % e.g. make_abs([(x : A), (y : B)], t, ?) unifies ? to lambda(x : A). lambda(y : B). t
601 make_abs(nil, Body, Body)
602 make_abs(cons(parameter(Name, Ty), Rest), Body, abs(Name, Ty, Acc_Abs)) :- make_abs(Rest, Body, Acc_Abs)
603
604 %% Constructs an app term from a function name and a vector of parameters to which
605 % the function is to be applied.

```

```

596 % e.g. make_app(f, [(x : A), (y : B)], ?) unifies to f x y
597 make_app(Fun, nil, Fun)
598 make_app(Fun, cons(parameter(Name, Ty), Rest), App) :- make_app(app(Fun, var(Name)), Rest, App)
599
600 %% Check module
601 check_module(Module) :- ctx.nil(Ctx) & check_module_with_context(Module, Ctx)
602
603 check_module_with_context(nil, Ctx)
604 check_module_with_context(cons(Declaration, Rest), Ctx) :- check_declaration(Declaration, Ctx, New_Ctx) &
605     check_module_with_context(Rest, New_Ctx)
606
607
608 %% Check axiom
609 check_declaration(axiom(Name, nil, Ty), Ctx, New_Ctx) :- infer(Ty, Ctx, Sort_Ty) & is_type(Sort_Ty) &
610     ctx.extend_axiom(Ctx, Name, Ty, New_Ctx)
611 check_declaration(axiom(Name, cons(Uni, Rest), Ty), Ctx, New_Ctx) :- infer(Ty, Ctx, Sort_Ty) &
612     is_type(Sort_Ty) & ctx.extend_def(Ctx, Name, cons(Uni, Rest), Ty, New_Ctx)
613
614 %% Check definition
615 check_declaration(def(Name, nil, Params, Ty, Body), Ctx, New_Ctx) :- ctx.extend_vars(Ctx, Params,
616     ↪ Uni_Var_Ctx) &
617     infer(Ty, Uni_Var_Ctx, Sort_Ty) & is_type(Sort_Ty) & make_pi_type(Params, Ty, Def_Ty) &
618     ctx.extend_axiom(Uni_Var_Ctx, Name, Def_Ty, Rec_Ctx) & make_abs(Params, Body, Lambda) &
619     check(Lambda, Def_Ty, Rec_Ctx) & ctx.extend_axiom(Ctx, Name, Def_Ty, New_Ctx)
620 check_declaration(def(Name, cons(Uni, Uni_Params), Params, Ty, Body), Ctx, New_Ctx) :-
621     ctx.extend_vars(Ctx, Params, Uni_Var_Ctx) & infer(Ty, Uni_Var_Ctx, Sort_Ty) & is_type(Sort_Ty) &
622     make_pi_type(Params, Ty, Def_Ty) & ctx.extend_axiom(Uni_Var_Ctx, Name, Def_Ty, Rec_Ctx) &
623     make_abs(Params, Body, Lambda) & check(Lambda, Def_Ty, Rec_Ctx) &
624     ctx.extend_def(Ctx, Name, cons(Uni, Uni_Params), Def_Ty, New_Ctx)
625
626 %% Check inductive
627 check_declaration(inductive(Name, nil, Params, Ty, Constrs), Ctx, New_Ctx) :-
628     ctx.extend_vars(Ctx, Params, Uni_Var_Ctx) & make_pi_type(Params, Ty, Ind_Ty) &
629     ctx.extend_axiom(Ctx, Name, Ind_Ty, Ext_Ctx) & ctx.extend_axiom(Uni_Var_Ctx, Name, Ind_Ty,
630     ↪ Ext_UV_Ctx) &
631     check_constrs(Constrs, Ext_Ctx, Ext_UV_Ctx, nil, Params, New_Ctx)
632 check_declaration(inductive(Name, cons(Uni, Uni_Params), Params, Ty, Constrs), Ctx, New_Ctx) :-
633     ctx.extend_vars(Ctx, Params, Uni_Var_Ctx) & make_pi_type(Params, Ty, Ind_Ty) &
634     ctx.extend_def(Ctx, Name, cons(Uni, Uni_Params), Ind_Ty, Ext_Ctx) &
635     ctx.extend_def(Uni_Var_Ctx, Name, cons(Uni, Uni_Params), Ind_Ty, Ext_UV_Ctx) &
636     check_constrs(Constrs, Ext_Ctx, Ext_UV_Ctx, cons(Uni, Uni_Params), Params, New_Ctx)
637
638 %% Check constructors
639 check_constrs(nil, Ctx, UV_Ctx, Uni_Params, Params, Ctx)
640 check_constrs(cons(constructor(Name, Ty), Rest), Ctx, UV_Ctx, nil, Params, New_Ctx) :-
641     infer(Ty, UV_Ctx, Constr_Sort) & is_type(Constr_Sort) & make_pi_type(Params, Ty, Constr_Ty) &
642     ctx.extend_constr(Ctx, Name, Constr_Ty, Inter_Ctx) & check_constrs(Rest, Inter_Ctx, UV_Ctx, nil,
643     ↪ Params, New_Ctx)
644 check_constrs(cons(constructor(Name, Ty), Rest), Ctx, UV_Ctx, cons(Uni, Uni_Params), Params, New_Ctx) :-
645     infer(Ty, UV_Ctx, Constr_Sort) & is_type(Constr_Sort) & make_pi_type(Params, Ty, Constr_Ty) &
646     ctx.extend_def(Ctx, Name, cons(Uni, Uni_Params), Constr_Ty, Inter_Ctx) &
647     check_constrs(Constrs, Ext_Ctx, Ext_UV_Ctx, cons(Uni, Uni_Params), Params, New_Ctx)
648
649 %% Check struct
650 %% TODO: we are not handling (universe) parameters properly here!
651 %% TODO: in general, structs with parameters do not typecheck properly since we
652 % are trying to unify the struct type with a pi type. This requires deeper thought.
653 check_declaration(struct(Name, nil, Params, Ty, Fields), Ctx, New_Ctx) :- ctx.extend_vars(Ctx, Params,
654     ↪ Params_Ctx) &
655     make_pi_type(Params, Ty, Struct_Ty) & ctx.extend_axiom(Ctx, Name, Struct_Ty, Ext_Ctx) &
656     check_fields(Name, Fields, Params, Param_Ctx, Ext_Ctx, New_Ctx)

```

```

653 check_declaration(struct(Name, cons(Uni, Uni_Params), Params, Ty, Fields), Ctx, New_Ctx) :-
654     ctx.extend_vars(Ctx, Params, Params_Ctx) & make_pi_type(Params, Ty, Struct_Ty) &
655     ctx.extend_def(Ctx, Name, cons(Uni, Uni_Params), Struct_Ty, Ext_Ctx) &
656     check_fields(Name, Fields, Params, Param_Ctx, Ext_Ctx, New_Ctx)
657
658 %% Check fields
659 check_fields(Struct_Name, nil, Params, Param_Ctx, Ctx, Ctx)
660 check_fields(Struct_Name, cons(field(Name, Ty), Rest), Params, Param_Ctx, Ctx, New_Ctx) :-
661     infer(Ty, Param_Ctx, Sort_Ty) & is_type(Sort_Ty) & make_app(var(Struct_Name), Params, App) &
662     make_pi_type(Params, pi("struct_param", App, Ty), Proj_Ty) & make_proj_name(Struct_Name, Name,
663     ↪ Proj_Name) &
664     ctx.extend_axiom(Ctx, Proj_Name, Proj_Ty, Inter_Ctx) & check_fields(Struct_Name, Rest, Params,
665     ↪ Param_Ctx, Inter_Ctx, New_Ctx)
666
667 %% Generate the name of the projection map for a field of a given struct.
668 % e.g. make_proj_name("struct", "field", ?) unifies ? to "struct.field"
669 make_proj_name(Struct_Name, Field_Name, Proj_Name) :- evaluate(stringappend(Struct_Name, "."),
670     ↪ Struct_Dot) &
671     evaluate(stringappend(Struct_Dot, Field_Name), Proj_Name)
672
673 %% Check terms
674 check(abs(X, Ty, Body), pi(Y, Expected_Ty, Expected_Body), Ctx) :- def_eq(Ty, Expected_Ty, Ctx) &
675     ctx.extend_var(Ctx, X, Ty, Ext_Ctx) & substitute(Y, var(X), Expected_Body, R_Expected_Body) &
676     check(Body, R_Expected_Body, Ext_Ctx)
677 check(Term, Expected_Ty, Ctx) :- ~is_abs(Term) & infer(Term, Ctx, Inferred_Ty) &
678     freshen_term(Inferred_Ty, Inferred_Fresh) & freshen_term(Expected_Ty, Expected_Fresh) &
679     def_eq(Inferred_Fresh, Expected_Fresh, Ctx)
680
681 %% Extract the universe variables from a universe.
682 get_uni_uvars(uconst(N), nil)
683 get_uni_uvars(uvar(X), [X])
684 get_uni_uvars(uadd(U, N), Uvars) :- get_uni_uvars(U, Uvars)
685 get_uni_uvars(umax(U, V), Uvars) :- get_uni_uvars(U, U_U) & get_uni_uvars(V, U_V) & union(U_U, U_V,
686     ↪ Uvars)
687
688 %% Extract the universe variables from a term.
689 get_uvars(var(X), [])
690 get_uvars(app(Fun, Arg), Uvars) :- get_uvars(Fun, U_Fun) & get_uvars(Arg, U_Arg) & union(U_Fun, U_Arg,
691     ↪ Uvars)
692 get_uvars(abs(X, Ty, Body), Uvars) :- get_uvars(Ty, U_Ty) & get_uvars(Body, U_Body) & union(U_Ty, U_Body,
693     ↪ Uvars)
694 get_uvars(pi(X, Ty, Body), Uvars) :- get_uvars(Ty, U_Ty) & get_uvars(Body, U_Body) & union(U_Ty, U_Body,
695     ↪ Uvars)
696 get_uvars(usort(U), Uvars) :- get_uni_uvars(U, Uvars)
697 get_uvars(match(Target, Arms), Uvars) :- get_uvars(Target, U_Target) & get_arms_uvars(Arms, U_Arms) &
698     ↪ union(U_Target, U_Arms, Uvars)
699 get_uvars(constructor(Name, Args), Uvars) :- get_args_uvars(Args, Uvars)
700 get_uvars(proj(Term, Field), Uvars) :- get_uvars(Term, Uvars)
701
702 get_arms_uvars(nil, nil)
703 get_arms_uvars(cons(match.arm(Pattern, Body), Rest), Uvars) :- get_uvars(Body, U_Body) &
704     ↪ get_arms_uvars(Rest, U_Rest) & union(U_Body, U_Rest, Uvars)
705
706 get_args_uvars(nil, nil)
707 get_args_uvars(cons(Arg, Rest), Uvars) :- get_uvars(Arg, U_Arg) & get_args_uvars(Rest, U_Rest) &
708     ↪ union(U_Arg, U_Rest, Uvars)
709
710 %% Instantiate a fresh metavariable for each distinct universe variable in a
711 % term. These must later be unified. Essential step for universe polymorphism.
712 freshen_term(Term, Fresh) :- get_uvars(Term, Uvars) & fresh_uni_params(Term, Uvars, Fresh)
713
714 %%%%%%%%%%%%%%

```

```

705 %%% Type Inference
706 %%%%%%%%%
707
708 %% Operations for type inference, required to deduce the type of a term from its
709 % structure and context.
710
711 %% Preprocessing for universe polymorphism.
712 fresh_uni_params(Term, Uni_Params, New_Term) :- make_fresh_uni_param_subst(Uni_Params, nil, S) &
713   ↪ uni_subst_all(Term, S, New_Term)
714
715 %% Creates a substitution for each universe parameter using a fresh metavariable.
716 make_fresh_uni_param_subst(nil, Substitution, Substitution)
717 make_fresh_uni_param_subst(cons(U, Rest), Substitution, New_Substitution) :-
718   map.insert(Substitution, U, Fresh_U, Inter_Substitution) &
719   make_fresh_uni_param_subst(Rest, Inter_Substitution, New_Substitution)
720
721 %% Infer the type of an application term from the pi type of the function and the
722 % inferred type of the argument.
723 infer_app(pi(X, Param_Ty, Body_Ty), Arg, Ctx, Type) :- infer(Arg, Ctx, Arg_Ty) & unify(Arg_Ty, Param_Ty)
724   ↪ & substitute(X, Arg, Body_Ty, Type)
725
726 %% Infer the universe level of a pi type.
727 % - Prop -> U has universe U
728 % - U -> Prop has universe U
729 % - U -> V for U, V != Prop has universe max(U, V)
730 infer_pi(uconst(0), U, Ctx, U)
731 infer_pi(U, uconst(0), Ctx, U)
732 infer_pi(U, V, Ctx, umax(U, V)) :- distinct(U, uconst(0)) & distinct(V, uconst(0))
733
734 %% Infer the universe level of a universe.
735 % - For a constant universe level N, this is simply the constant N+1 (computed statically)
736 % - For any other universe U, this is U+1 (non-static)
737 infer_uni(uconst(N), uconst(M)) :- evaluate(plus(N, 1), M)
738 infer_uni(U, uadd(U, 1)) :- ~is_uconst(U)
739
740 %% Infer variable type.
741 infer(var(X), Ctx, Type) :- ctx.lookup_var(Ctx, X, Type)
742 infer(var(X), Ctx, New_Ty) :- ~ctx.contains_var(Ctx, X) & ctx.lookup_def(Ctx, X, ctx.def(Uni_Params, Ty))
743   ↪ & fresh_uni_params(Ty, Uni_Params, New_Ty)
744 infer(var(X), Ctx, Type) :- ~ctx.contains_var(Ctx, X) & ~ctx.contains_def(Ctx, X) & ctx.lookup_axiom(Ctx,
745   ↪ X, Type)
746 infer(var(X), Ctx, Type) :- ~ctx.contains_var(Ctx, X) & ~ctx.contains_def(Ctx, X) &
747   ↪ ~ctx.contains_axiom(Ctx, X) & ctx.lookup_constr(Ctx, X, Type)
748
749 %% Infer application type.
750 infer(app(var(Name), Arg), Ctx, Type) :- ctx.lookup_def(Ctx, Name, ctx.def(Uni_Params, Ty)) &
751   ~empty(Uni_Params) & infer(constructor(Name, [Arg]), Ctx, Type)
752
753 infer(app(Fun, Arg), Ctx, Type) :- infer(Fun, Ctx, Fun_Ty) & infer_app(Fun_Ty, Arg, Ctx, Type)
754
755 %% Infer abstraction type.
756 infer(abs(X, Ty, Body), Ctx, pi(X, Ty, Body_Ty)) :- infer(Ty, Ctx, Sort_Ty) & is_type(Sort_Ty) &
757   ctx.extend_var(Ctx, X, Ty, Ext_Ctx) & infer(Body, Ext_Ctx, Body_Ty)
758
759 %% Infer pi type.
760 infer(pi(X, Ty, Body), Ctx, usort(Nf_Pi_Uni)) :- infer(Ty, Ctx, usort(Ty_Uni)) & ctx.extend_var(Ctx, X,
761   ↪ Ty, Ext_Ctx) &
762   infer(Body, Ext_Ctx, usort(Body_Uni)) & infer_pi(Ty_Uni, Body_Uni, Ctx, Pi_Uni) &
763   ↪ universe.normalize(Pi_Uni, Nf_Pi_Uni)
764
765 %% Infer usort type.
766 infer(usort(U), Ctx, usort(Nf_U)) :- infer_uni(U, R_U) & universe.normalize(R_U, Nf_U)

```

```

760
761
762 %% Infer constructor type.
763 infer(constructor(Name, Args), Ctx, R_Ty) :- ctx.lookup_constr(Ctx, Name, Ty) & infer_constr_args(Args,
    ↪ Ty, Ctx, R_Ty)
764 infer(constructor(Name, Args), Ctx, R_Ty) :- ~ctx.contains_constr(Ctx, Name) &
765     ctx.lookup_def(Ctx, Name, ctx.def(Uni_Params, Ty)) & fresh_uni_params(Ty, Uni_Params, New_Ty) &
766     infer_constr_args(Args, New_Ty, Ctx, R_Ty)
767
768 infer_constr_args(nil, Constr_Ty, Ctx, Constr_Ty)
769 infer_constr_args(cons(Arg, Rest), pi(Param, Param_Ty, Body_Ty), Ctx, R_Ty) :- infer(Arg, Ctx, Arg_Ty) &
770     def_eq(Arg_Ty, Param_Ty, Ctx) & substitute(Param, Arg, Body_Ty, R_Body_Ty) & infer_constr_args(Rest,
    ↪ R_Body_Ty, Ctx, R_Ty)
771
772
773 %% Infer match type.
774 infer(match(Target, Arms), Ctx, Arms_Ty) :- infer(Target, Ctx, Target_Ty) & infer_arms(Arms, Target_Ty,
    ↪ Ctx, Arms_Ty)
775
776 infer_arms([match.arm(Pattern, Body)], Target_Ty, Ctx, Arm_Ty) :- check_pattern(Pattern, Target_Ty, Ctx,
    ↪ Pattern_Ctx) &
777     infer(Body, Pattern_Ctx, Arm_Ty)
778 infer_arms(cons(match.arm(Pattern, Body), Rest), Target_Ty, Ctx, Arm_Ty) :-
    ↪ infer_arms([match.arm(Pattern, Body)], Target_Ty, Ctx, Arm_Ty) &
779     infer_arms(Rest, Target_Ty, Ctx, Rest_Ty) & def_eq(Arm_Ty, Rest_Ty, Ctx)
780
781 check_pattern(pattern.var(X), Expected_Ty, Ctx, New_Ctx) :- ctx.lookup_constr(Ctx, X, Ty) &
782     check_pattern(pattern.constructor(X, nil), Ctx, Expected_Ty, New_Ctx)
783 check_pattern(pattern.var(X), Expected_Ty, Ctx, New_Ctx) :- ~ctx.contains_constr(Ctx, X) &
    ↪ ctx.extend_var(Ctx, X, Expected_Ty, New_Ctx)
784 check_pattern(pattern.wildcard, Expected_Ty, Ctx, Ctx)
785 check_pattern(pattern.constructor(Name, Patterns), Expected_Ty, Ctx, New_Ctx) :- ctx.lookup_constr(Ctx,
    ↪ Name, Ty) &
786     spec_constr_ty(Ty, Expected_Ty, Spec_Ty) & get_constr_out(Spec_Ty, Out_Ty) & def_eq(Out_Ty,
    ↪ Expected_Ty, Ctx) &
787     get_constr_args(Spec_Ty, Pattern_Tys) & check_all_patterns(Patterns, Pattern_Tys, Ctx, New_Ctx)
788 check_pattern(pattern.constructor(Name, Patterns), Expected_Ty, Ctx, New_Ctx) :-
    ↪ ~ctx.contains_constr(Ctx, Name) &
789     ctx.lookup_def(Ctx, Name, ctx.def(Uni_Params, Ty)) & fresh_uni_params(Ty, Uni_Params, New_Ty) &
    ↪ get_constr_out(New_Ty, Out_Ty) &
790     unify(Out_Ty, Expected_Ty) & get_constr_args(New_Ty, Pattern_Tys) & check_all_patterns(Patterns,
    ↪ Pattern_Tys, Ctx, New_Ctx)
791
792 check_all_patterns(nil, nil, Ctx, Ctx)
793 check_all_patterns(cons(Pattern, Rest), cons(Pattern_Ty, Rest_Ty), Ctx, New_Ctx) :-
    ↪ check_pattern(Pattern, Pattern_Ty, Ctx, Inter_Ctx) &
794     check_all_patterns(Rest, Rest_Ty, Inter_Ctx, New_Ctx)
795
796 %% Specialize the type of a constructor from the types of its arguments.
797 spec_constr_ty(Ty, Expected_Ty, Ty) :- get_pi_type_params(Ty, nil)
798 spec_constr_ty(Ty, Expected_Ty, Spec_Ty) :- get_pi_type_params(Ty, cons(Type_Param, Rest)) &
799     get_app_args(Expected_Ty, Type_Args) & apply_spec(Ty, cons(Type_Param, Rest), Type_Args, Spec_Ty)
800
801 %% Extract the type parameters from a pi type.
802 get_pi_type_params(pi(X, usort(U), Body), cons(X, Type_Params)) :- get_pi_type_params(Body, Type_Params)
803 get_pi_type_params(pi(X, Ty, Body), nil) :- ~is_usort(Ty)
804 get_pi_type_params(Ty, nil) :- ~is_pi(Ty)
805
806 %% Extract the arguments from an application.
807 get_app_args_rev(app(Fun, Arg), cons(Arg, Type_Args)) :- get_app_args_rev(Fun, Type_Args)
808 get_app_args_rev(Term, nil) :- ~is_app(Term)
809

```

```

810 get_app_args(Term, Args) :- get_app_args_rev(Term, Rev_Args) & rev(Rev_Args, Args)
811
812 %% Specialize a given type using the provided type arguments.
813 apply_spec(Ty, nil, Type_Args, Ty)
814 apply_spec(pi(Ty_Param, Ty, Body), cons(Ty_Param, Rest_Param), cons(Ty_Arg, Rest_Arg), Spec_Body) :-
815   ↪ substitute(Ty_Param, Ty_Arg, Body, Sub_Body) & apply_spec(Sub_Body, Rest_Param, Rest_Arg, Spec_Body)
816 apply_spec(pi(X, Ty, Body), cons(Ty_Param, Rest_Param), cons(Ty_Arg, Rest_Arg), Spec_Ty) :- distinct(X,
817   ↪ Ty_Param) & apply_spec(pi(X, Ty, Body), Rest_Param, Rest_Arg, Spec_Ty)
818 apply_spec(Ty, cons(Ty_Param, Rest_Param), cons(Ty_Arg, Rest_Arg), Spec_Ty) :- ~is_pi(Ty) &
819   ↪ apply_spec(Ty, Rest_Param, Rest_Arg, Spec_Ty)
820
821 %% Extract the codomain type of a constructor.
822 get_constr_out(pi(X, Ty, Body), Out_Ty) :- get_constr_out(Body, Out_Ty)
823 get_constr_out(Constr_Ty, Constr_Ty) :- ~is_pi(Constr_Ty)
824
825 %% Extract the (non-type) parameter types of a constructor.
826 get_constr_args(pi(X, usort(U), Body), Arg_Tys) :- get_constr_args(Body, Arg_Tys)
827 get_constr_args(pi(X, Ty, Body), cons(Ty, Arg_Tys)) :- ~is_usort(Ty) & get_constr_args(Body, Arg_Tys)
828 get_constr_args(Constr_Ty, nil) :- ~is_pi(Constr_Ty)
829
830 %% Extract the structure name and number of arguments from a struct application.
831 % This is required for parameterized structs.
832 get_struct_from_app(var(Struct), Struct, 0)
833 get_struct_from_app(app(Fun, Arg), Struct, N) :- get_struct_from_app(Fun, Struct, M) & evaluate(plus(M,
834   ↪ 1), N)
835
836 %% Find the parameters of a struct in the context.
837 collect_struct_parameters(Struct, Ctx, Struct_Params) :- ctx.lookup_axiom(Ctx, Struct, Struct_Ty) &
838   ↪ get_pi_params(Struct_Ty, Struct_Params)
839 collect_struct_parameters(Struct, Ctx, Struct_Params) :- ~ctx.contains_axiom(Ctx, Struct) &
840   ↪ ctx.lookup_def(Ctx, Struct, ctx.def(Uni_Params, Struct_Ty)) & get_pi_params(Struct_Ty, Struct_Params)
841
842 %% Extract all the parameters of an iterated pi type.
843 get_pi_params(pi(X, Ty, Body), cons(parameter(X, Ty), Rest)) :- get_pi_params(Body, Rest)
844 get_pi_params(Term, nil) :- ~is_pi(Term)
845
846 %% TODO: this is the previous implementation of type inference for projection
847 % terms. It is currently deprecated, since it cannot handle structs with
848 % parameters. However, the current implementation of infer for projections below
849 % is experimental, so until it has been fully tested this should be left alone
850 % for more ease of testing.
851 %%%infer(proj(Term, Field), Ctx, Cod_Ty) :- infer(Term, Ctx, Ty) & get_struct_from_app(Ty, Struct_Name,
852   ↪ Num_Params) &
853 %%%   make_proj_name(Struct_Name, Field, Proj_Name) & ctx.lookup_axiom(Ctx, Proj_Name, pi(X, Dom_Ty,
854   ↪ Cod_Ty)) &
855 %%%   def_eq(Dom_Ty, var(Struct_Name), Ctx)
856
857 %% Procedure for projection type inference with parameterized structs:
858 % 1. break down app(S, A) into the struct name S and the arguments [Arg1, ...] = [A,...].
859 % 2. make the projection name S.f
860 % 3. look up S.f, get back pi(Param1, Ty1, pi(Param2, Ty2, ..., pi(ParamN, TyN, Expected_Proj_Pi)))
861 % 4. look up S in axioms/defs, and collect the parameters of S. Strip that many pis from the type of S.f
862   ↪ to get Expected_Proj_Pi
863 % 5. substitute Parami -> Argi in Expected_Proj_Pi to form Proj_Ty
864 % 6. break down Proj_Ty = pi(struct_param, Dom, Cod)
865 % 7. expect Dom = app(S, Arg1, ..., ArgN)
866 % 8. Infer type as Cod
867
868 %% Apply a pi type to a given argument term.
869 apply_pi(pi(X, Ty, Body), Term, Ctx, Sub_Body) :- check(Term, Ty, Ctx) &
870   ↪ substitute(X, Term, Ty, Sub_Ty) & substitute(X, Term, Body, Sub_Body)
871
872

```

```

864
865 %% Apply an iterated pi type to a list of argument terms.
866 apply_pi_all(Ty, nil, Ctx, Ty)
867 apply_pi_all(Pi_Ty, cons(Arg, Rest), Ctx, Sub_Ty) :- apply_pi(Pi_Ty, Arg, Ctx, Inter_Ty) &
868   ↪ apply_pi_all(Inter_Ty, Rest, Ctx, Sub_Ty)
869
870 %% Break an iterated application into the function (struct) name and a list of
871 % its arguments.
872 decouple_struct_args(var(Struct), Struct, nil)
873 decouple_struct_args(app(Struct_Fun, Arg), Struct, cons(Arg, Args)) :- decouple_struct_args(Struct_Fun,
874   ↪ Struct, Args)
875
876 %% Type inference for projection terms. Currently (sort of) handles structs
877 % with parameters, but universe polymorphism fails in certain cases. TODO.
878 infer(proj(Term, Field), Ctx, Cod_Ty) :- infer(Term, Ctx, Ty) & decouple_struct_args(Ty, Struct_Name,
879   ↪ Struct_Args) & ctx.lookup_axiom(Ctx, Struct_Name, Struct_Ty) &
880   make_proj_name(Struct_Name, Field, Proj_Name) & ctx.lookup_axiom(Ctx, Proj_Name, Proj_Ty) &
881   apply_pi_all(Struct_Ty, Struct_Args, Ctx, Sub_Struct_Ty) & apply_pi_all(Proj_Ty, Struct_Args, Ctx,
882     ↪ pi(X, Dom_Ty, Cod_Ty)) &
883   def_eq(Dom_Ty, Sub_Struct_Ty, Ctx)
884
885 infer(proj(Term, Field), Ctx, Cod_Ty) :- infer(Term, Ctx, Ty) & decouple_struct_args(Ty, Struct_Name,
886   ↪ Struct_Args) & ~ctx.contains_axiom(Ctx, Struct_Name, Struct_Ty) &
887   ctx.lookup_def(Ctx, Struct_Name, ctx.def(Uni_Params, Struct_Ty)) & make_proj_name(Struct_Name, Field,
888     ↪ Proj_Name) &
889   ctx.lookup_axiom(Ctx, Proj_Name, Proj_Ty) & fresh_uni_params(Struct_Ty, Uni_Params, Fresh_Ty) &
890   apply_pi_all(Fresh_Ty, Struct_Args, Ctx, Sub_Struct_Ty) & apply_pi_all(Proj_Ty, Struct_Args, Ctx,
891     ↪ pi(X, Dom_Ty, Cod_Ty)) &
892   fresh_uni_params(Dom_Ty, Uni_Params, Fresh_Dom_Ty) & def_eq(Fresh_Dom_Ty, Sub_Struct_Ty, Ctx)

```